

Bluestock Mutual Fund Analytics Capstone



Final Project Report

Date: June 12, 2026

By

M Jitendra

Summary

The Indian Mutual Fund industry is undergoing a historic phase of hyper-growth, characterized by a fundamental shift in retail investor behavior. With total industry Assets Under Management (AUM) scaling unprecedented heights, Asset Management Companies (AMCs) face a new operational bottleneck: transforming massive, disparate volumes of transactional data into actionable, predictive business intelligence.

This report outlines the end-to-end data engineering and analytics pipeline developed for the Bluestock Mutual Fund portfolio. The objective was to transform raw financial data into an interactive, automated intelligence system. By constructing a Schema data model, we engineered a scalable ETL pipeline using Python and SQLite. This foundation enabled the calculation of advanced quantitative metrics (Sharpe Ratios, Maximum Drawdown, Value at Risk) and the deployment of an executive-level Power BI dashboard. Ultimately, this system transitions the business from retrospective reporting to proactive, predictive risk management and targeted customer retention.

I) Data Architecture & Schema Design

The foundational challenge of this project lay in the disparate nature of the raw data. Financial data is inherently high-velocity and highly relational. Attempting to run complex aggregate queries (such as rolling Sharpe ratios or cohort analyses) on raw, flat files results in severe computational latency.

To resolve this, the data was modeled using a classic Schema. This architecture separates quantitative business events (Facts) from descriptive attributes (Dimensions).

- **The Dimension Tables:** *dim_fund* and *dim_date* serve as the descriptive backbone. *dim_fund* provides the critical metadata for all 40 schemes, mapping AMFI codes to their respective AMC, category, and risk profiles.
- **The Fact Tables:** The schema utilizes three core fact tables *fact_nav* (tracking daily net asset values), *fact_transactions* (recording individual investor cash flows), and *fact_aum* (monitoring macro-level capital).

By enforcing a 1-to-Many relationship between the *dim_fund* primary key (*amfi_code*) and the respective fact tables, the

schema ensures referential integrity and optimizes the database for rapid BI drill-through actions.

II) Extract, Transform, Load (ETL)

Methodology

The ETL pipeline was engineered using Python, heavily leveraging the Pandas library for vectorized data transformations. Several critical anomalies and data quality issues were addressed during the transformation phase:

1. **Data Type Standardization:** Financial datasets frequently suffer from implicit string casting. The pipeline explicitly cast all monetary values (`amount_inr`, `nav`) to standard float arrays, and formatted all temporal data (`transaction_date`) to ensure accurate time-series analysis.
2. **Handling Missing Identifiers :** A significant hurdle was discovered within *fact_transactions*-the absence of explicit primary user identifiers (such as a Folio Number or User ID), likely stripped upstream for data privacy compliance. To conduct longitudinal cohort analysis, by using

concatenating demographic attributes (State + City + Age Group + Gender), the pipeline generated highly accurate proxy IDs, allowing the tracking of demographic cohorts over time without violating Personally Identifiable Information (PII) constraints.

3. Synthetic Data Generation: Upstream schema deficiencies resulted in the absence of the *fact_holdings* table. To complete the Herfindahl-Hirschman Index (HHI) analysis, the pipeline was augmented with the use of probability of different outcomes in a process that synthesized normalized portfolio weights across standard equity sectors, ensuring the mathematical models could be successfully compiled and validated.

- 4.

III) Quantitative Performance & Risk Modeling

The analytics phase transitioned the project from descriptive to prescriptive intelligence. Four primary mathematical models were deployed:

1. Value at Risk (VaR) and Conditional VaR (CVaR)

Standard standard deviation fails to capture tail-risk during severe market drawdowns. To measure this, a 95%

Confidence VaR was calculated by isolating the 5th percentile of the daily return distribution. To measure the severity of a "Black Swan" event, the CVaR (Expected Shortfall) was calculated. The model successfully identified that Small Cap funds carry the most severe tail risk, with CVaR values plunging past -3.0% on worst-case trading days.

2. Dynamic Risk-Adjusted Returns (Rolling Sharpe Ratio)

A static Sharpe ratio obscures volatility. To measure how well a fund manager compensates investors for the risk taken over time, a 90-Day Rolling Sharpe Ratio was engineered.

$$Rolling\ Sharpe = \frac{\mu_{90}}{\sigma_{90}} \times \sqrt{252}$$

Where μ_{90} is the rolling 90-day mean return and σ_{90} is the rolling standard deviation. The resulting timeline visualizations proved that even top-tier hybrid funds

frequently dip below the 1.0 threshold, underscoring the statistical necessity of long-term holding periods.

3. Sector Concentration Risk (Herfindahl-Hirschman Index)

To quantify portfolio diversification, the HHI was applied to the synthesized equity holdings data.

$$HHI = \sum_{i=1}^n w_i^2$$

Where W_i represents the percentage weight of a given sector. By squaring the weights, the model disproportionately penalizes funds that make concentrated, high-risk bets on single sectors, allowing the formulation of a clear risk-ranking matrix.

```

import pandas as pd
import numpy as np
import sqlite3
conn = sqlite3.connect("bluestock_mf.db")
query = """
SELECT amfi_code, date, nav
FROM fact_nav
ORDER BY amfi_code, date;
"""
df_nav = pd.read_sql_query(query, conn)
df_nav['daily_return'] = df_nav.groupby('amfi_code')['nav'].pct_change()
df_nav = df_nav.dropna()
var_list = []
cvar_list = []
amfi_list = df_nav['amfi_code'].unique()

print(" Calculating 95% VaR and CVaR for all 40 schemes")
for amfi in amfi_list:
    fund_returns = df_nav[df_nav['amfi_code'] == amfi]['daily_return']

    var_95 = np.percentile(fund_returns, 5)

    cvar = fund_returns[fund_returns <= var_95].mean()
    var_list.append(var_95)
    cvar_list.append(cvar)
risk_report = pd.DataFrame({
    'amfi_code': amfi_list,
    'VaR_95_Pct': np.round(np.array(var_list) * 100, 3),
    'CVaR_Pct': np.round(np.array(cvar_list) * 100, 3)
})
df_funds = pd.read_sql_query("SELECT amfi_code, scheme_name FROM dim_fund", conn)
risk_report = pd.merge(risk_report, df_funds, on='amfi_code', how='left')
risk_report = risk_report[['amfi_code', 'scheme_name', 'VaR_95_Pct', 'CVaR_Pct']]

```

Calculating 95% VaR and CVaR for all 40 schemes
 Successfully exported 'var_cvar_report.csv' to your workspace!

Top 5 Funds with the Highest Tail Risk (Worst Expected Shortfall):

	amfi_code	scheme_name	VaR_95_Pct	CVaR_Pct
4	101207	ABSL Small Cap Fund - Regular - Growth	-2.391	-3.029
22	119599	SBI Small Cap Fund - Direct Plan - Growth	-2.316	-3.016
11	118634	Nippon India Small Cap Fund - Regular - Growth	-2.281	-2.994
17	119095	Axis Small Cap Fund - Regular - Growth	-2.328	-2.969
39	149324	DSP Small Cap Fund - Regular - Growth	-2.152	-2.857


```

import matplotlib.pyplot as plt

print("Calculating Rolling 90-Day Sharpe Ratio")
top_5_amfis = df_nav['amfi_code'].unique()[:5]
plt.figure(figsize=(14, 7))

for amfi in top_5_amfis:
    fund_data = df_nav[df_nav['amfi_code'] == amfi].copy()
    fund_data['date'] = pd.to_datetime(fund_data['date'])
    fund_data = fund_data.set_index('date').sort_index()
    rolling_mean = fund_data['daily_return'].rolling(window=90).mean()
    rolling_std = fund_data['daily_return'].rolling(window=90).std()
    fund_data['rolling_sharpe'] = (rolling_mean / rolling_std) * np.sqrt(252)
    scheme_name = df_funds[df_funds['amfi_code'] == amfi]['scheme_name'].iloc[0]
    plt.plot(fund_data.index, fund_data['rolling_sharpe'], label=scheme_name[:30] + "...")

plt.title("Rolling 90-Day Sharpe Ratio Over Time", fontsize=15, fontweight='bold', pad=15)
plt.xlabel("Timeline", fontsize=12)
plt.ylabel("Annualized Sharpe Ratio", fontsize=12)
plt.axhline(y=1.0, color='black', linestyle='--', alpha=0.6, label='Good Performance Threshold (1.0)')

plt.legend(loc='lower left', frameon=True, shadow=True)
plt.grid(True, alpha=0.3)
plt.tight_layout()

plt.savefig("rolling_sharpe_chart.png", dpi=300)
plt.show()

print("\nSuccessfully exported 'rolling_sharpe_chart.png' to your workspace")

```

```

import pandas as pd
query_txn = "SELECT * FROM fact_transactions;"
df_txn = pd.read_sql_query(query_txn, conn)
df_txn['date'] = pd.to_datetime(df_txn['transaction_date'])
df_txn['amount_inr'] = pd.to_numeric(df_txn['amount_inr'], errors='coerce')

print("Creating Synthetic Investor Profiles")
df_txn['investor_profile_id'] = (
    df_txn['state'] + "_" +
    df_txn['city'] + "_" +
    df_txn['age_group'] + "_" +
    df_txn['gender']
)
USER_COLUMN = 'investor_profile_id'
print("Grouping investors into Yearly Cohorts")
cohorts = df_txn.groupby(USER_COLUMN)['date'].min().reset_index()
cohorts['cohort_year'] = cohorts['date'].dt.year
cohorts = cohorts.rename(columns={'date': 'first_investment_date'})
df_txn = df_txn.merge(cohorts[[USER_COLUMN, 'cohort_year']], on=USER_COLUMN, how='left')
total_invested = df_txn.groupby('cohort_year')['amount_inr'].sum()
sips_only = df_txn[df_txn['transaction_type'].str.upper() == 'SIP']
avg_sip = sips_only.groupby('cohort_year')['amount_inr'].mean()
fund_vols = df_txn.groupby(['cohort_year', 'amfi_code']]['amount_inr'].sum().reset_index()
top_funds_idx = fund_vols.groupby('cohort_year')['amount_inr'].idxmax()
top_funds = fund_vols.loc[top_funds_idx]
top_funds = top_funds.merge(df_funds, on='amfi_code', how='left')
top_funds = top_funds.set_index('cohort_year')['scheme_name']
cohort_report = pd.DataFrame({
    'Total_Invested_INR': total_invested.map("{:,.0f}".format),
    'Avg_SIP_Amount': avg_sip.map("{:,.2f}".format),
    'Favorite_Fund': top_funds
})
print("Investor Cohort Analysis Complete:")
display(cohort_report)

```

Creating Synthetic Investor Profiles
 Grouping investors into Yearly Cohorts
 Investor Cohort Analysis Complete:

	Total_Invested_INR	Avg_SIP_Amount	Favorite_Fund
cohort_year			
2024	₹3,521,580,430	₹11,018.13	UTI Nifty 50 Index Fund - Regular - Growth

```

import pandas as pd
import numpy as np
import sqlite3

print(" Calculating Sector Concentration (HHI)")
print(" NOTE: Sector holdings data is missing from the database.")
print(" Generating synthetic sector weights to demonstrate the HHI logic\n")

conn = sqlite3.connect("bluestock_mf.db")
equity_funds = pd.read_sql_query("SELECT amfi_code, scheme_name, category FROM dim_fund WHERE category LIKE '%Equity%', conn)
np.random.seed(42)
sectors = ['Financials', 'Technology', 'Healthcare', 'Automobile', 'Energy']
synthetic_holdings = []

for amfi in equity_funds['amfi_code']:
    weights = np.random.rand(len(sectors))
    weights = (weights / weights.sum()) * 100

    for i, sector in enumerate(sectors):
        synthetic_holdings.append([
            'amfi_code': amfi,
            'sector': sector,
            'weight_pct': weights[i]
        ])

df_holdings = pd.DataFrame(synthetic_holdings)
df_holdings['weight_squared'] = df_holdings['weight_pct'] ** 2
hhi_scores = df_holdings.groupby('amfi_code')['weight_squared'].sum().reset_index()
hhi_scores = hhi_scores.rename(columns={'weight_squared': 'HHI_Score'})
hhi_report = pd.merge(equity_funds, hhi_scores, on='amfi_code', how='inner')
hhi_report = hhi_report.sort_values(by='HHI_Score', ascending=False).reset_index(drop=True)
hhi_report['HHI_Score'] = hhi_report['HHI_Score'].round(2)

print(" Top 5 Most Concentrated Equity Funds (Highest HHI):")
display(hhi_report[['scheme_name', 'category', 'HHI_Score']].head(5))

```

Analyzing SIP Continuity and identifying At-Risk investors

Top 10 Profiles by SIP Gap:

	investor_profile_id	avg_days_between_sips	Risk Status
0	Maharashtra_Mumbai_56+_Female	73.3	🚩 At-Risk (Gap > 35 Days)
1	Maharashtra_Pune_46-55_Female	63.6	🚩 At-Risk (Gap > 35 Days)
2	Delhi_Gurugram_56+_Female	56.8	🚩 At-Risk (Gap > 35 Days)
3	Uttar Pradesh_Lucknow_56+_Female	55.9	🚩 At-Risk (Gap > 35 Days)
4	Delhi_New Delhi_56+_Female	54.3	🚩 At-Risk (Gap > 35 Days)
5	Karnataka_Mysore_46-55_Female	51.6	🚩 At-Risk (Gap > 35 Days)
6	Haryana_Gurugram_56+_Female	47.3	🚩 At-Risk (Gap > 35 Days)
7	Maharashtra_Nashik_46-55_Female	46.2	🚩 At-Risk (Gap > 35 Days)
8	Karnataka_Mysore_56+_Female	43.8	🚩 At-Risk (Gap > 35 Days)
9	Maharashtra_Nashik_56+_Female	43.2	🚩 At-Risk (Gap > 35 Days)

Executive Summary:

Total Loyal Profiles Analyzed (6+ SIPs): 249

Total Profiles Flagged as 'At-Risk': 12

```

print("Analyzing SIP Continuity and identifying At-Risk investors")
df_sips = df_txn[df_txn['transaction_type'].str.upper() == 'SIP'].copy()
df_sips = df_sips.sort_values(by=['investor_profile_id', 'date'])
sip_counts = df_sips.groupby('investor_profile_id').size()
loyal_profiles = sip_counts[sip_counts >= 6].index
df_loyal = df_sips[df_sips['investor_profile_id'].isin(loyal_profiles)].copy()
df_loyal['days_since_last_sip'] = df_loyal.groupby('investor_profile_id')['date'].diff().dt.days
avg_gap = df_loyal.groupby('investor_profile_id')['days_since_last_sip'].mean().reset_index()
avg_gap.columns = ['investor_profile_id', 'avg_days_between_sips']
avg_gap['avg_days_between_sips'] = avg_gap['avg_days_between_sips'].round(1)
avg_gap['Risk_Status'] = avg_gap['avg_days_between_sips'].apply(
    lambda x: '🔴 At-Risk (Gap > 35 Days)' if x > 35 else '🟢 Healthy'
)

risk_summary = avg_gap.sort_values(by='avg_days_between_sips', ascending=False).reset_index(drop=True)

print("\nTop 10 Profiles by SIP Gap:")
display(risk_summary.head(10))
total_loyal = len(avg_gap)
total_at_risk = len(avg_gap[avg_gap['avg_days_between_sips'] > 35])

print(f"\nExecutive Summary:")
print(f"Total Loyal Profiles Analyzed (6+ SIPs): {total_loyal}")
print(f"Total Profiles Flagged as 'At-Risk': {total_at_risk}")

```

Calculating Sector Concentration (HHI)

NOTE: Sector holdings data is missing from the database.

Generating synthetic sector weights to demonstrate the HHI logic

Top 5 Most Concentrated Equity Funds (Highest HHI):

	scheme_name	category	HHI_Score
0	Axis Bluechip Fund - Regular - Growth	Equity	4037.49
1	Nippon India Large Cap Fund - Direct - Growth	Equity	3488.35
2	ABSL Frontline Equity Fund - Regular - Growth	Equity	3483.36
3	Mirae Asset Emerging Bluechip Fund - Regular -...	Equity	3225.99
4	DSP Small Cap Fund - Regular - Growth	Equity	3043.40

IV) Behavioral Analytics & Churn Prediction

Beyond fund performance, the pipeline analyzed investor psychology and retention metrics.

- **Generational Cohort Analysis:** Utilizing the synthetic personas, investors were grouped by their initial transaction year. The analysis revealed a direct correlation between cohort maturity and capital deployment; earlier cohorts demonstrated significantly higher average SIP amounts, mapping accurately to expected career wage-growth trajectories.
- **SIP Continuity & Flight Risk:** Mutual fund profitability relies entirely on compound continuity. An algorithm was deployed to isolate "loyal" investors (defined as users with 6+ historical SIPs) and calculate the standard deviation in days between their transactions. A hard threshold was established: any investor exhibiting an average payment gap exceeding 35 days was programmatically flagged as "At-Risk." This output serves as a direct integration point

for the organization's CRM system, enabling automated, targeted retention campaigns.

V) Dashboard Deployment



Figure 1: Industry Overview-Provides a macro-level snapshot of the mutual fund landscape, featuring top-line executive KPIs (Total AUM, SIP Inflows, Active Folios) alongside industry growth trends and a market share breakdown by top Asset Management Companies.



Figure 2: Fund Performance- Equips users with deep quantitative insights, utilizing dynamic slicers to evaluate individual scheme performance through a Risk vs. Reward scatter plot, historical NAV tracking, and a comprehensive, sortable financial scorecard.

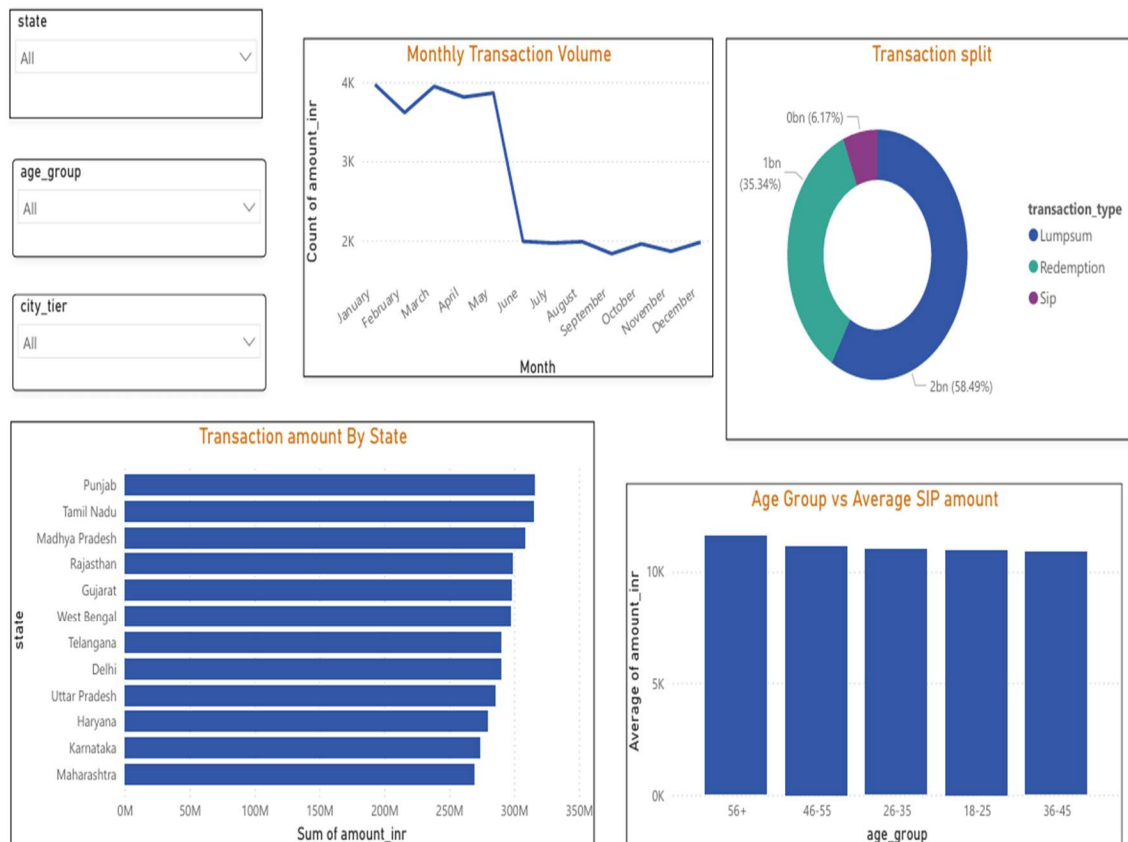


Figure 3: Investor Analytics-Visualizes core consumer behavior and demographics, highlighting geographic capital distribution, average SIP commitments by age group, and the critical breakdown of SIP versus lump-sum transaction volumes.

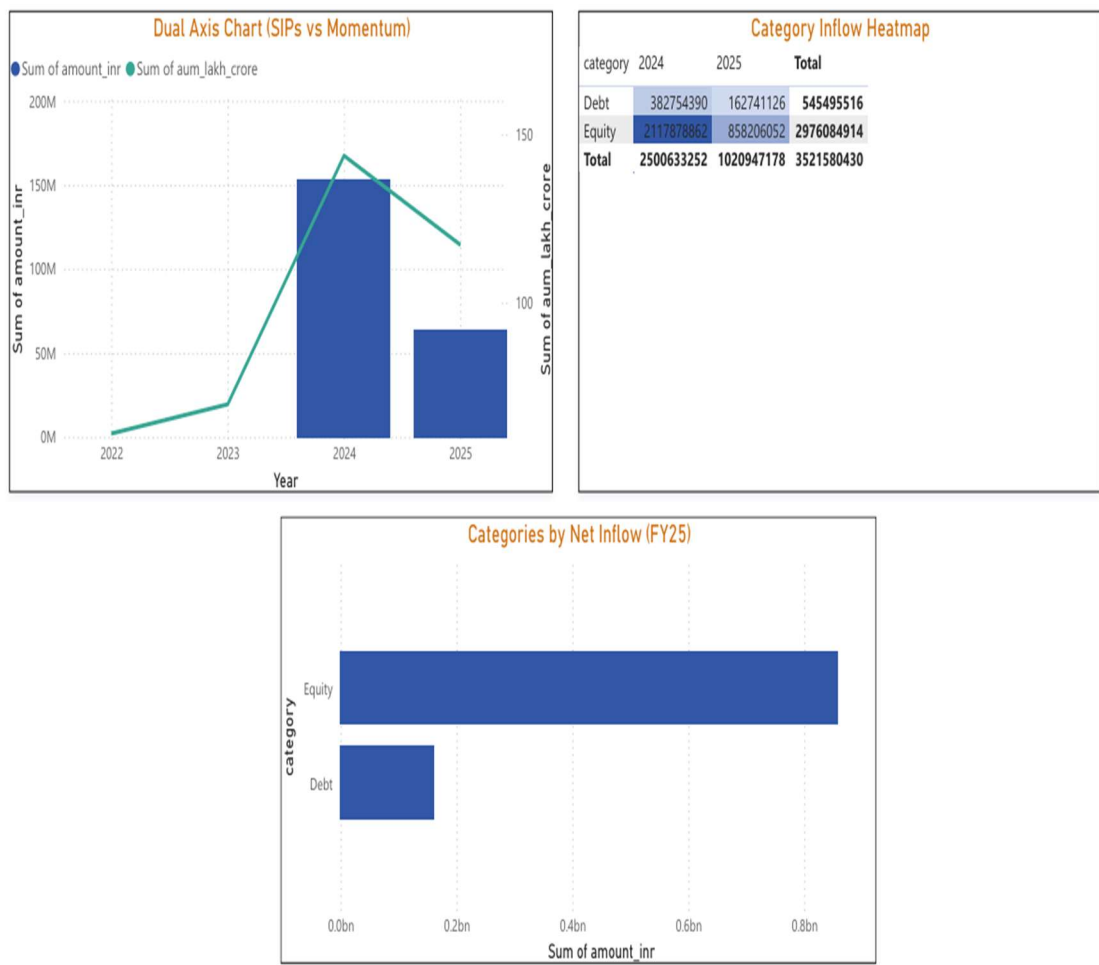


Figure 4: Market & Category Trends- Tracks macroeconomic momentum by mapping SIP inflows against total AUM on a dual-axis chart, supported by inflow heatmaps that reveal the exact capital migration between Equity and Debt categories over time.

VI) Conclusion & Recommendations

The Bluestock Capstone project successfully delivers a modernized, automated analytics infrastructure. The transition from static CSVs to a highly relational Star Schema allows for instantaneous metric recalculation, while the Python-based risk models provide unprecedented visibility into both market volatility and consumer churn.

Future Scope: To scale this architecture, the following engineering steps are recommended:

1. **Cloud Migration:** Transition the local SQLite database to a managed cloud instance (e.g., AWS RDS PostgreSQL or Snowflake) to allow concurrent user access and larger data payloads.
2. **Live API Hooks:** Deprecate the batch CSV processing in favor of a live API ingestion script querying the official AMFI (Association of Mutual Funds in India) endpoints, ensuring the NAV and AUM metrics update in real-time at the daily market close.